

Intermediate (build real memory) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Work through in order — don't jump ahead until you've genuinely tried.

- [Hint 1](#)
- [Hint 2](#)
- [Hint 3](#)
- [Hint 4](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 1

Simple Version

There's no built-in "memory" to turn on. You build memory yourself, by keeping a growing list of every message so far, and sending the *whole* list again every time you call the model.

Start with a list that has just your system message in it. Each time around your loop: add the user's new message to the list, send the whole list, get a reply, add the reply to the list too.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

The whole exercise is one idea: a Python list, mutated in place, that grows by exactly two entries per turn — one `{"role": "user", ...}` and one `{"role": "assistant", ...}`. Every single call sends this entire list, not just the newest message; the model has no server-side memory of its own (this is the exact "No memory" idea from Doc03).

Structure it as a loop: read input, append it as a user message, call the API with the full history, append the assistant's reply back into the same list, print it, and loop again.

Sketch the loop's structure (in plain English, no code yet) before moving to Hint 2.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 2

Simple Version

Watch out for this exact bug: forgetting to append the assistant's reply back into the history list. If you skip that step, the model never sees its own previous answers — it'll look like it "forgot" everything within a couple of turns, and it's the single most common mistake here.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

The response object doesn't give you back a ready-made message dict — you build one yourself from the reply text: `{"role": "assistant", "content": response.choices[0].message.content}`. Append this to `history`, not just print it, or the next turn's call won't include it.

Also decide how the loop ends — a sentinel input like typing `"quit"` is the simplest approach: check for it right after reading input, before appending anything, so the exit command itself never becomes part of the conversation history sent to the model.

Write the loop's body now, including both appends (user, then assistant) before checking Hint 3.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 3

Simple Version

```
history = [system message]

loop forever:
    read user_input
    if user_input is "quit": stop the loop

    add {user, user_input} to history
    send history to the model, get reply
    add {assistant, reply} to history
    print reply
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
history = [
    {"role": "system", "content": "You are a helpful assistant."}
]

while True:
    user_input = input("You: ")
    if user_input == "quit":
        break

    history.append({"role": "user", "content": user_input})

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    reply = response.choices[0].message.content

    history.append({"role": "assistant", "content": reply})
    print("Assistant:", reply)
```

Notice `history` is never rebuilt from scratch — it's the exact same list object, growing by two entries every loop iteration. Trace through 2-3 turns by hand and confirm `history` has the length you expect after each one.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 4

Simple Version

To actually prove memory works, ask something in one turn that only makes sense if the model remembers the previous turn — like telling it your name in turn 1, then asking "what's my name?" in turn 2. If it answers correctly, memory is working.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

A good test sequence: turn 1 — "My favorite color is teal." turn 2 — "What's my favorite color?" If `history` is being built and sent correctly, turn 2's answer references teal. If you get a generic "I don't have access to that information" answer, check first whether you actually appended the assistant's reply from turn 1 back into `history` — that's the bug from Hint 2 showing up in practice.

Run this test now, then check your loop against the full [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Full solution: [Show me the solution](#)